

CS444 A1 Design Document

Scanner, Parser, AST, and Weeder

Arnav Tripathi (a22tripa)

Ryan Maxin (rsmxin)

Andreja Japundzic (ajapundz)

Design and Implementation	3
Overview	3
Creating Scanner DFA	4
Exception Handling	4
Choice of Ocaml for Implementation	4
Grammar	5
Tree Visualization	5
Abstractor	5
Testing and Known Issues	6
Testing	6
Known Issues	7
Contributions	7
Thoughts	7
Appendix	9
1. Test Suite Output	9
2. AST for Simple program (.dot GraphViz diagram)	10
3. Parse tree for Simple Program (.dot GraphViz diagram)	11
4. Master NFA	12

Design and Implementation

Overview

Our compiler is organized into four main components, as per the specification in the assignment: the scanner, the parser, the AST builder (the “abstractor”), and the weeder. These components are arranged as a sequence of independent front-end stages executed in chronological order, each exposing a narrow interface to the next stage.

1. **Driver (bin/main.ml | bin/driver.ml):** The main entry point of the compiler. It reads the input Joos files and invokes each stage of the compilation pipeline in order. The driver wraps the entire execution in a try/catch block to manage error propagation, assigning distinct error codes to each stage. It returns exit code 42 on failure to comply with the Marmoset testing harness.
2. **Scanner module (lib/frontend/scanner):** Reads a Joos source file and applies maximal munch recursively using a hardcoded DFA to produce a list of Joos tokens.
3. **Parser module (lib/frontend/parser):** Uses the defined .lalr1 grammar (generated with the provided parse table generator) to construct data structures representing the valid-prefix DFA and its production rules, along with the corresponding action and goto hash tables. These structures are then used to perform the LALR(1) shift-reduce parsing algorithm and construct the parse tree.
4. **AST module (lib/frontend/abstractor):** First performs basic canonicalization on the parse tree (removing redundant nodes, flattening lists, and discarding wrapper nonterminals), along with some pre-weeding that is more naturally performed on the raw parse tree. It then converts the parse tree into a tree composed of specialized AST type definitions that enforce structural invariants of the language grammar.
5. **Weeder module (lib/frontend/weeder):** AST traversals that enforce non-grammar constraints.
6. **Shared types (lib/types):** This directory contains all object definitions/constructions that these interfaces may need to share, such as tokens, exceptions, parse_tree nodes, AST definitions, etc.
7. **Utility/helper functions (lib/utilities):** Contains reusable helper functions shared across the compiler, such as decode_char_literal for handling escape sequences. It also provides tree visualization utilities that generate .dot files during parsing, which can be rendered with Graphviz to visualize the parse tree.

Creating Scanner DFA

For our scanner, the first design decision we had to make with our implementation was whether we were going to attempt to implement our own Lex, or create a DFA by hand that would define all the states/transitions we would need. From the recommendation of the professor and reading through the *Modern Compiler Implementation in ML* textbook’s lexing section, we decided to go with the latter approach. To us, it was an efficient way to start making progress on our scanner work as it allowed us to parallelize our work by having one individual work on creating the DFA/encoding it in Ocaml, and another individual begin the implementation of the scanner with a dummy DFA.

To implement our DFA, we built independent graphs using the .dot format (which can be easily compiled into a visual graph), and unioned them together using epsilon transitions (see our master NFA in

appendix). From here we performed the epsilon closure by hand to build our lib/frontend/scanner/dfa.ml file.

Our dfa.ml file is 277 lines, which we believe was a good tradeoff of manual work rather than attempting to get a working Lex implementation. However, a prominent struggle we ran into was when issues surfaced with lexing, we had to tweak our hardcoded dfa.ml file constantly in order to fix bad transitions. Some of these errors even surfaced after we had seemingly moved on from lexing, into the parsing stages.

Exception Handling

A pivotal decision of our implementation that paid dividends when it came to debugging/testing was having rigorous exception handling. The boundaries we had placed for the different modules allowed us to isolate errors that would show up in our pipeline by stage. We were able to create exception types specific to lexing/parsing/weeding, add custom messages that would highlight exactly what went wrong, as well as tracing values of any relevant variables.

Choice of Ocaml for Implementation

The choice to use Ocaml as our programming language was largely inspired from this [article](#) by Dwight VandenBerghe, which pinpoints the key attributes of meta-languages that make it suitable for compiler construction compared to other general-purpose programming languages. Our compiler front-end is dominated by transformations over tree-structured data (tokens, parse trees, and weed checks). As VandenBerghe argues, ML-family languages are well-suited to compilers because algebraic data types and pattern matching make AST/token representations explicit and difficult to misuse, while strong static typing and type inference allow rapid iteration without sacrificing safety. In our implementation, this showed up directly: tokens and parse-tree nodes are modeled as variant types, and most compiler logic becomes recursive structure-driven code (e.g., reductions in the shift-reduce parser and weed passes over the tree). OCaml's garbage collection also removed an entire class of memory-management complexity that we would have faced in C++ when constructing and discarding many short-lived nodes during scanning/parsing. Finally, OCaml's exceptions and module system fit our "phase-separated" design: each phase can fail with a clear error type, and internal representations are hidden behind module interfaces so that changes to one phase (e.g., DFA transitions or parse-tree node constructors) don't cascade through the codebase.

Grammar

The grammar was designed using the Java 1 and Java 2 specification along with the Joos1W feature website. It was a long process that was a test of attention to detail more than anything else. When designing the grammar, Java 1 was the baseline as we knew it was LR(1). Every grammar rule in Java 1 was cross checked with the Java 2 grammar and the Joos1W feature list to determine if it should be included as is, slightly changed, or removed altogether. In the end, we got an LR(1) grammar that is mostly a skimmed down version of the Java 1 grammar with a few augmentations for Joos1W.

To highlight some examples, all the grammar rules that removed ambiguity in the Java 1 grammar were used, such there being a general modifier nonterminal used by instances, classes, constructors, etc. We later in the weeder checked whether the right modifiers were being used. Additionally, the grammar was changed to not allow the initialization of multi-dimensional arrays, not allow array declaration after variable name, and not allow array data ({1, 2, 3, 4} after the initializer).

Spending a considerable amount of time on the grammar saved us a lot of time during the parsing as we just had to implement the parsing algorithm. It also saved us a lot of time on the weeding as we were able to cover certain rules in the grammar without having to check them in the weeder.

Tree Visualization

We spent time adding functions which would output our parse trees and ASTs to .dot formats. This was hugely important for understanding the output of our parser and for helping us decide how to canonicalize and build our abstract syntax tree. It also helped immensely for figuring out how to implement certain rules within the weeder. In the appendix you can find the visualizations of a parse tree using our visualizer for a simple program and then the AST for that same program.

Abstractor

For our AST construction phase, we focused on three primary goals: canonicalization, pre-weeding, and abstraction.

The canonicalizer performs basic transformations on the parse tree to reduce its size and simplify later processing. This includes removing redundant nodes (such as parentheses that are no longer needed after parsing), flattening list structures to avoid deep recursive traversals (e.g., formal parameter lists), and eliminating wrapper nodes that always contain a single child.

Next, we perform pre-weeding. We introduced this stage after realizing that certain semantic constraints rely on information present in the raw parse tree but lost during abstraction. Specifically, we handle two cases during pre-weeding: distinguishing type casts and checking for INT_MIN represented as a parenthesized integer. Both cases depend on detecting redundant parentheses, which would otherwise be removed during canonicalization and abstraction.

Finally, the abstractor maps the many specific grammar rules into a more generalized and structured AST. For example, instead of maintaining distinct node types for each binary operator, we introduce a single binop node type parameterized by the operator. Each AST node is defined as its own type, allowing us to enforce structural constraints directly through the type system.

Testing and Known Issues

Testing

Similarly to our phased implementation, we separated our test suite into sections for the three main components of scanning, parsing, and weeding. Before we moved onto the implementation of the next stage of the pipeline, we made sure our outputs matched our expectations.

For the scanner: our test suite in `test_suite/scanner` included special edge cases such as unterminated block comments, sequences of accepted/rejected escape characters, differentiating between equals vs assign operators.

For the parser, since we decided to use the test suite provided to us by the assignment. Our thought process was that if we have a successful parse, then we have the guarantee that all tests prefixed with J1 should be accepted and match the expected output of 0. For the tests prefixed with Je, we would have to differentiate between tests that were caused by actual errors in our parsing, or weeding errors that our compiler would not catch at that point.

We created a bash script `test_suite/a1/test_a1.sh` that feeds in the contents of these test cases to our `./joosc` executable as files, and it also checks the prefix of these filenames to denote what the expected output should be. The script runs and gives us a count of all the passing/failing tests. Specifically for the error cases, instead of outputting the 0/42 values for success and failure respectively, we logged the error message of our exceptions to pinpoint exactly what part of our compiler was rejecting, once again portraying the importance of our exception handling.

Known Issues

As of 02/11/2026, there are no known issues with the implementation of our scanner/parser/weeder. We have a 100% pass rate on the public test cases provided to us, and we also have included some edge cases for scanning and parsing present in the `test-suite/` directory that match the intended behaviour.

Contributions

Arnav:

- Implemented the original logic for the scanner (maximal munch, scan function).
- Cleaned up the first iteration of parsing logic to remove redundant code and add more traceability.
- Added first `test_suite` for some scanner tests and wrote the initial bash script to read in provided test cases.
- Outlined and completed the design document

Andreja:

- Setup the Ocaml project
- Defined the tokens for the tokenizer
- Built the individual DFAs for the scanner (i.e. String Literal DFA)
- Defined the grammar used by the parser and built the DFA using the given program
- Added Octal escapes and did the Weeding with Ryan

Ryan:

- Unified individual DFAs into a master NFA $\rightarrow$ DFA and encoded the scanner in OCaml; built a scanner tester and ensured all test cases passed.
- Established project structure and Makefile; cleaned and stabilized scanner implementation.
- Implemented parser infrastructure: aligned `.cfg` with tokens, built `.lalr1` loader, parsing algorithm, parse tree construction, and supporting data structures.

- Developed the AST-building framework (abstractor), optimized its implementation, and built an AST visualizer.
- Reorganized and expanded the test suite, collaborated on debugging/weeding with Andreja, and added additional edge-case tests for A1.

Thoughts

Working through this assignment gave us an appreciation for how important engineering discipline is when making a compiler. Before we began writing code for each separate stage of our implementation, we made sure to have a fundamental understanding of all the requirements and made markdown files that we could easily reference for our implementation. Defining clear invariants between phases, manually drawing our master NFA/DFA in .dot GraphViz notation, and building debugging infrastructure were pivotal to the completion of this project in a timely manner.

Overall, we spent a considerable amount of time on this assignment. Every week we had three meetings; one on Monday and Friday as a quick check in and one on Wednesday to do a group working session. Organization was key, especially with the nature of the compiler where each part depends on the previous part. We had to ensure that we were completing each part efficiently while also looking ahead to start the next part. In the end, it was a lot of hard work, but it was very satisfying to see all the parts come together to pass all the test cases.

In terms of difficulty, this assignment was quite challenging. There was an extension given which allowed us to complete the project without sacrificing too much sleep, but it was definitely a tight timeline. As a group we were consistently well organized with this compiler being our top priority and yet we were still just able to get it done in time. Maybe setting the timeline to be the Friday before reading week would have allowed for just a bit more breathing room, but the extension to Wednesday was an overall huge relief.

We really liked the overall design of the assignment. It allowed for a lot of freedom in choosing how to implement the compiler, which we feel provided a lot of learning opportunities which you normally wouldn't get with stricter guidelines. Some parts were more tedious than others, like building the DFA for the scanning by hand, but we do understand that it was done this way to allow us to learn the process in depth.

For our implementation on Ocaml, the language features that benefitted us most were the case analysis features provided by the *match* keyword, strong type checking to guarantee invariants, and the functional nature of the language, which made it intuitive to work with the tree data structures that are prevalent in compiler programming. However, some of the drawbacks during the debugging stage were adding side effects to certain functions. Trying to add log statements or pretty printing when a function is returning a non-unit type led to erroneous syntax that took time to fix. Since OCaml makes sequencing explicit, we sometimes had to restructure code using `let () = ... in` or `;` to insert debug output without changing return types, and small mistakes there led to confusing type/syntax errors.

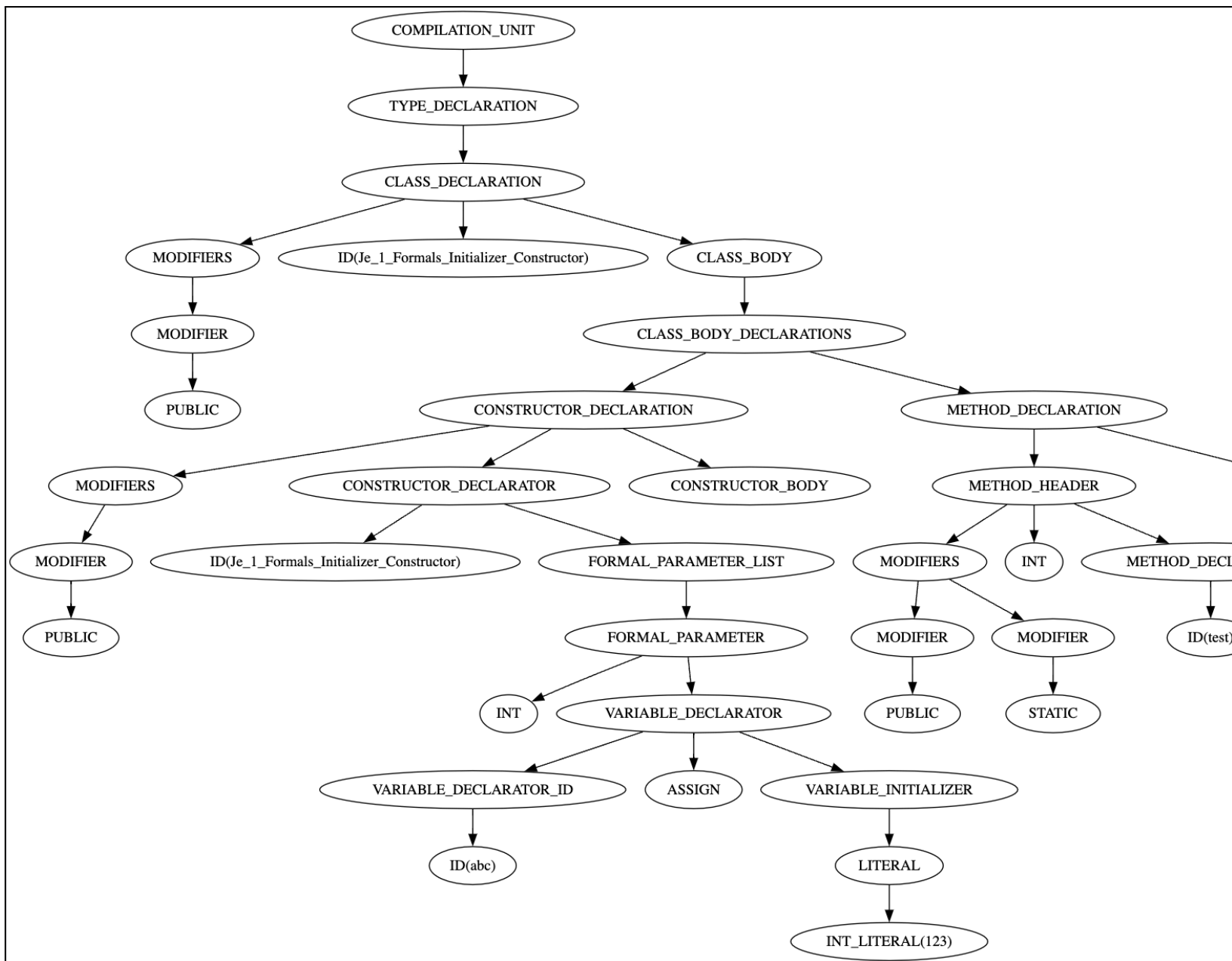
Appendix

1. Test Suite Output

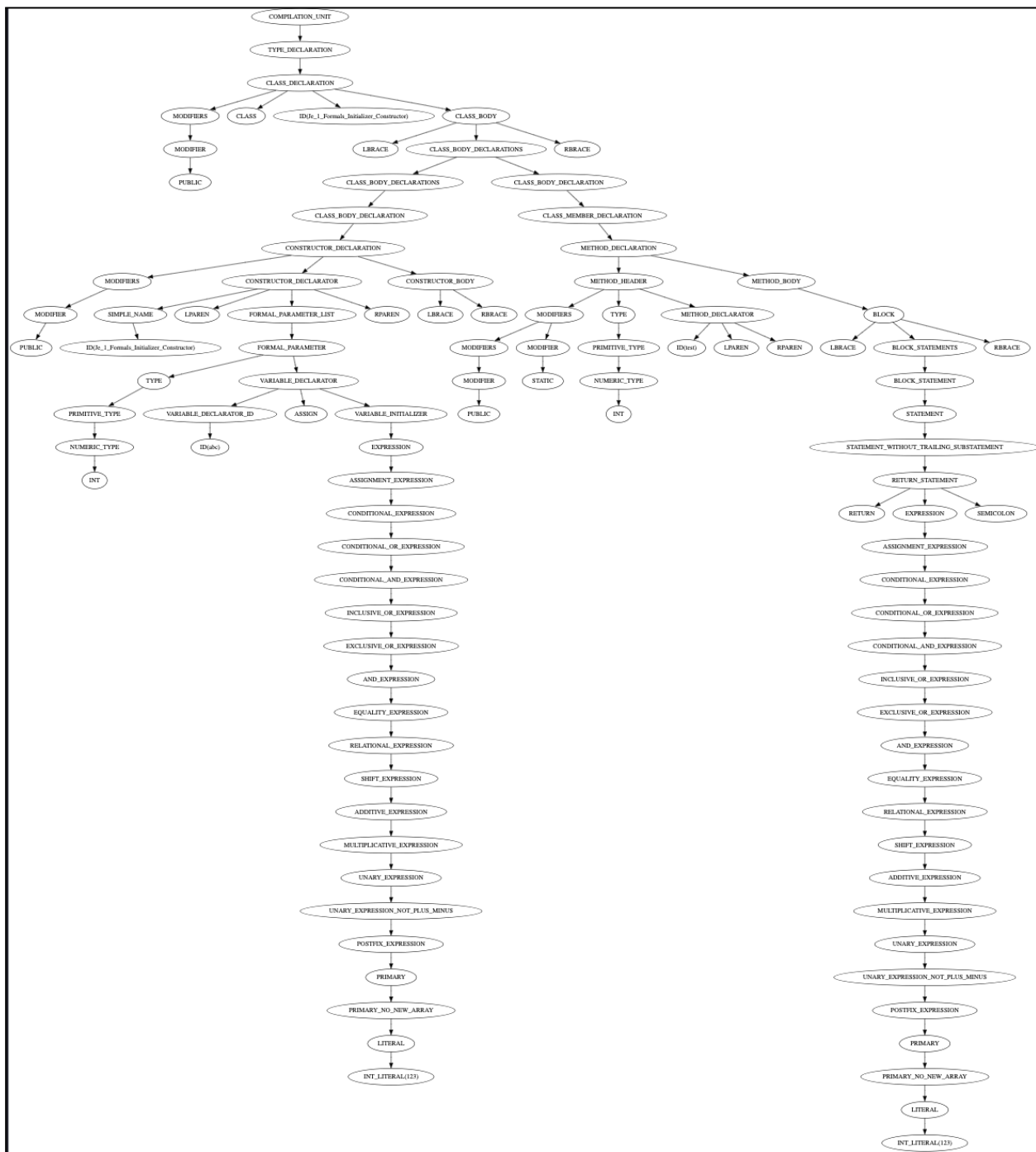
```
✓ Je_1 NonJoosConstructs_EmptyConstructor.java (error=PARSE_ERROR)
✓ Je_1 NonJoosConstructs_MultipleTypesPrFile.java (error=PARSE_ERROR)
✓ Je_1 NonJoosConstructs_NestedTypes.java (error=PARSE_ERROR)
✓ Je_1 NonJoosConstructs_PrivateFields.java (error=LEX_ERROR)
✓ Je_1 NonJoosConstructs_PrivateMethods.java (error=LEX_ERROR)
✓ Je_1 NonJoosConstructs_StaticInitializers.java (error=PARSE_ERROR)
✓ Je_1 NonJoosConstructs_Strictftp.java (error=LEX_ERROR)
✓ Je_1 NonJoosConstructs_SuperMethodCall.java (error=LEX_ERROR)
✓ Je_1 NonJoosConstructs_Switch.java (error=LEX_ERROR)
✓ Je_1 NonJoosConstructs_Synchronized.java (error=LEX_ERROR)
✓ Je_1 NonJoosConstructs_SynchronizedStatement.java (error=LEX_ERROR)
✓ Je_1 NonJoosConstructs_Transient.java (error=LEX_ERROR)
✓ Je_1 NonJoosConstructs_UnaryPlus.java (error=PARSE_ERROR)
✓ Je_1 NonJoosConstructs_Unicode.java (error=LEX_ERROR)
✓ Je_1 NonJoosConstructs_Volatile.java (error=LEX_ERROR)
✓ Je_1 PackagePrivate_Class.java (error=WEEDEDER_ERROR)
✓ Je_1 PackagePrivate_Field.java (error=WEEDEDER_ERROR)
✓ Je_1 PackagePrivate_Method.java (error=WEEDEDER_ERROR)
✓ Je_1 SuperThis_SuperAfterBlock.java (error=LEX_ERROR)
✓ Je_1 SuperThis_SuperAfterStatement.java (error=LEX_ERROR)
✓ Je_1 SuperThis_SuperInBlock.java (error=LEX_ERROR)
✓ Je_1 SuperThis_SuperInMethod.java (error=LEX_ERROR)
✓ Je_1 SuperThis_SuperThis.java (error=LEX_ERROR)
✓ Je_1 SuperThis_ThisAfterStatement.java (error=PARSE_ERROR)
✓ Je_1 SuperThis_ThisInMethod.java (error=PARSE_ERROR)
✓ Je_1 SuperThis_TwoSuperCalls.java (error=LEX_ERROR)
✓ Je_1 Throw_NotExpression.java (error=LEX_ERROR)
✓ Je_1 Throws_Array.java (error=LEX_ERROR)
✓ Je_1 Throws_SimpleType.java (error=LEX_ERROR)
✓ Je_1 Throws_Void.java (error=LEX_ERROR)
✓ Je_1 VoidType_ArrayCreation.java (error=PARSE_ERROR)
✓ Je_1 VoidType_ArrayDeclaration.java (error=PARSE_ERROR)
✓ Je_1 VoidType_Cast.java (error=PARSE_ERROR)
✓ Je_1 VoidType_Field.java (error=PARSE_ERROR)
✓ Je_1 VoidType_Formals.java (error=PARSE_ERROR)
✓ Je_1 VoidType_Local.java (error=PARSE_ERROR)
✓ Je_1 VoidType_VoidMethod.java (error=PARSE_ERROR)
✓ Je_6 Assignable_Instanceof_SimpleTypeOfSimpleType.java (error=PARSE_ERROR)
✓ Je_6 InstanceOf_Primitive_1.java (error=PARSE_ERROR)
✓ Je_6 InstanceOf_Primitive_2.java (error=PARSE_ERROR)
✓ Je_MissingExplicitConstructor.java (error=WEEDEDER_ERROR)
✓ Je_Native.java (error=WEEDEDER_ERROR)
✓ Je_Throws.java (error=LEX_ERROR)
✓ J1_Subfolder_NameMatch.java

=====
Summary:
  Total:  327
  Passed: 327
  Failed:  0
All tests passed!
○ arnavt800@DESKTOP-LOHTQNC:~/cs444/joos1w-compiler$
```

2. AST for Simple program (.dot GraphViz diagram)



3. Parse tree for Simple Program (.dot GraphViz diagram)



4. Master NFA

Master NFA - Epsilon Transitions to All Sub-DFAs

